



LIORA CAPSTONE PROJECT PROPOSAL

AiDemy Pub

A demand-driven pipeline that chooses school topics, generates checked learning material and explainer videos, and publishes them on YouTube after one human click.

Track LLMOps - Track A (Workflow LLM)

Difficulty 6/10

Build time 8 weeks, about 15 working days

Author Anastasiya

THE QUESTION THE SYSTEM ANSWERS

Which topics should AiDemy publish next, and is the generated material correct enough to publish?

01 Detailed description

The goal of this project is to build a system that chooses which school topics to produce, generates a German learning pack (PDF) and two explainer videos for each chosen topic, checks the content, and uploads the videos to YouTube after one human click. The whole chain is maintained by an automated end-to-end pipeline.

AiDemy creates learning material for German Gymnasium students (curriculum of Saxony). Two working prototypes already exist: a learning pack generator (LLM, audit-and-repair loop, PDF renderer) and a video generator that turns the same lesson file (*bundle.json*) into a long video (16:9) and a YouTube Short (9:16).

Today both are run by hand, one topic at a time. The topic choice is a guess, nothing is versioned or monitored, and the videos are not checked.

The emphasis is on the production chain, not on model performance. The existing generators are reused as they are, behind a stable interface. The work of this project is everything around them: API, containers, versioning, orchestration, quality checks, monitoring and deployment.

02 Scope of this version

The project is deliberately small, so that the complete chain can be built in about 8 weeks next to the course, and every module of the programme is still covered.

In scope

- Geschichte, grades 5 and 6 (about 40-50 topics)
- YouTube only: long video and Short, private until approved
- PDF stored and ready; selling it stays manual

Out of scope

- TikTok, Instagram
- Online stores (Payhip, Etsy, eduki)
- Other subjects, maths formulas, other languages

Existing components (reused, not rewritten)

- **Learning pack generator** (`generate_lesson.py`): topic in, `bundle.json` and PDF out, with a built-in audit-and-repair loop.
- **Video generator** (`main.py --bundle`): `bundle.json` in, long video, Short and SEO metadata out.
- **Topic lists** for the Saxony curriculum with seed SEO keywords.

Both generators are called as command-line tools. The link between them is the `bundle.json` file, which gets a fixed schema. This keeps the integration work small and safe.

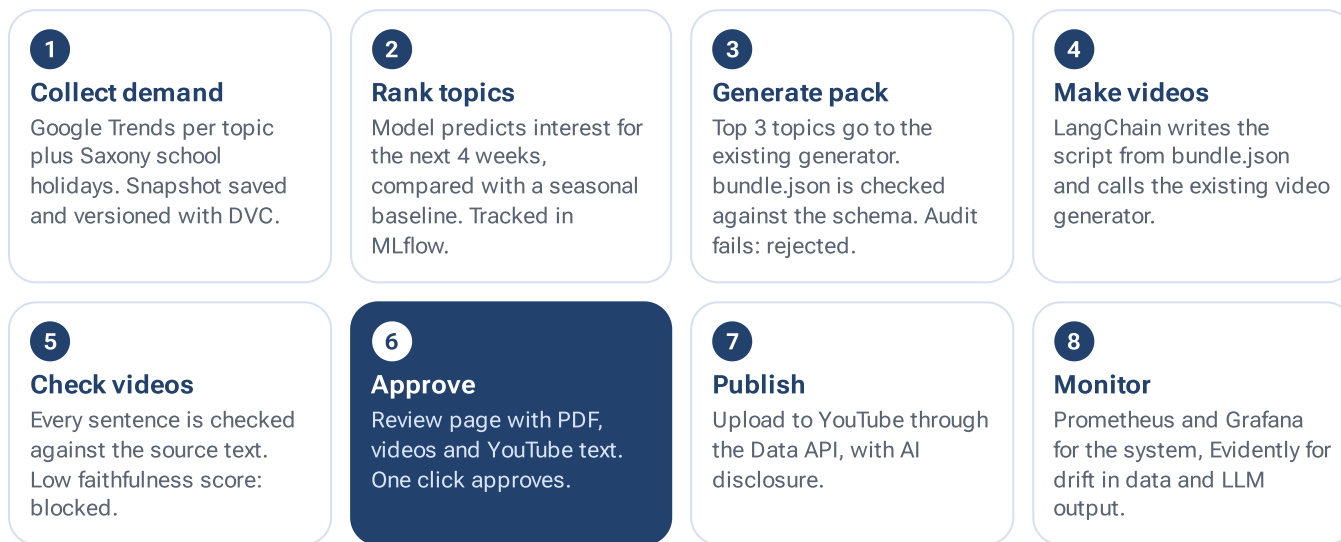
03 Coverage of the course requirements

Phase	Required	How this project covers it
1 - Prompting week 8	Workflow use case, Docker environment, prompt engineering, FastAPI, objectives and metrics	Use case and metrics document, docker compose, prompts in versioned files, FastAPI endpoints for ranking, generation and videos
2 - Build week 12	MLflow tracking, LangChain chains, workflow pipeline, DVC, NGINX	MLflow for model and generation runs, two LangChain chains, end-to-end pipeline script, DVC for data, NGINX in front of all services
3 - Deployment week 18	Airflow orchestration, LLM API serving, LLM monitoring, MinIO	Weekly Airflow DAG, LLM workflow served through FastAPI, LLM metrics (tokens, latency, errors, cost), MinIO for artifacts

Phase	Required	How this project covers it
4 - Monitoring week 23	Prometheus and Grafana, prompt optimization, full observability, Evidently, Localstack, Kubernetes	One Grafana dashboard, two prompt versions compared in MLflow, Evidently drift report, API on minikube, Localstack S3
All phases	GDPR, security, ethics, Git and GitHub, agile work	GDPR section below, GitHub with pull requests and CI, Kanban board

04 How the system works

Every Monday the chain runs by itself. A human only approves the result.



The dark step is the only manual one. Any failed check stops the chain for that topic (fail-closed).

Components and tools

Component	Tools
API and containers	FastAPI, Docker, docker compose, NGINX
LLM workflow	LangChain (video script, faithfulness check), one small LLM client with retry, fallback and cost logging, prompts in versioned files
Data and artifacts	DVC (topic list, demand snapshots), MinIO (bundles, PDFs, videos, models), Localstack S3 in the cloud setup
Tracking	MLflow: demand model runs and registry, one run per generated topic (prompt version, model, tokens, cost, quality results)
Orchestration	Airflow: one weekly DAG for the whole chain, one DAG for publishing
Demand model	scikit-learn (gradient boosting), seasonal baseline
Monitoring	Prometheus, Grafana, Evidently
Deployment	Kubernetes (minikube) for the API, Localstack, GitHub Actions (tests and image build)

05 Project phases

Phase 1 - Prompting and foundations

about 4 days

WHAT TO DO

- Write the use case, objectives and metrics.
- One GitHub repository (api, pipelines, demand, dags, infra). Remove secrets from the code and git history.
- Define the bundle.json schema (pydantic) with tests: the contract between the two generators.
- Move the prompts into versioned files.
- Docker images for the API and both generators, started with docker compose.
- FastAPI: GET /topics/ranking, POST /bundles, POST /videos, GET /health.

HOW IT WILL WORK

"docker compose up" starts everything. One request creates the learning pack for a topic, a second one creates the videos.

Deliverables: objectives and metrics, Docker environment, REST API.

Phase 2 - Build

about 4 days

WHAT TO DO

- Demand data for 40-50 topics from Google Trends (up to 5 years of weekly history, so the model can be trained at once), plus school holidays. Versioned with DVC.
- Demand model: seasonal baseline plus one gradient boosting model. Runs in MLflow, best model in the registry.
- MLflow run for every generated topic: prompt version, model, tokens, cost, duration, audit result.
- LangChain chains for the video script and the faithfulness check.
- NGINX in front of the API, MLflow and the review page.

HOW IT WILL WORK

One script runs the whole chain, from ranking to checked videos. Every result can be traced back to its data, prompt version and model.

Deliverables: MLflow tracking and registry, LangChain workflow, DVC, NGINX.

Phase 3 - Deployment

about 4 days

WHAT TO DO

- Airflow DAG "weekly_content": collect demand, rank, generate the top 3 topics, make and check the videos, put them in the approval queue.
- Airflow DAG "publish": upload approved videos to YouTube.
- MinIO for all artifacts.
- Review page (simple HTML served by FastAPI) with Approve and Reject.
- LLM monitoring: the LLM client exposes Prometheus metrics (tokens, latency, errors, cost per call).

HOW IT WILL WORK

Every Monday the chain runs by itself. When the material is ready, it waits on the review page. One click publishes the videos.

Deliverables: Airflow orchestration, LLM API serving, LLM monitoring, MinIO.

Phase 4 - Monitoring and production

about 3-4 days

WHAT TO DO

- One Grafana dashboard: pipeline success, run duration, LLM cost and latency, quality gate pass rate, faithfulness score, rejection rate.
- Weekly Evidently report: drift in demand data and in LLM output (faithfulness score, text length).
- Prompt optimization: two versions of the video script prompt compared on 10 topics in MLflow. The better one is promoted.
- Kubernetes: API and model serving on minikube. Heavy generators stay in docker compose.
- Localstack S3 for the cloud setup (same S3 interface as MinIO, only the endpoint changes).
- GitHub Actions: tests and image build on every push.

HOW IT WILL WORK

Problems such as a broken prompt, a rising cost or a change in demand are visible on one dashboard and in the weekly drift report.

Deliverables: full observability, drift detection, prompt optimization, Kubernetes and Localstack.

Defense (week 24). Live demo in 20 minutes: start the weekly DAG for 2-3 topics, show why they were ranked first, show the runs in MLflow, open the PDF and the Short, click Approve, show the private upload on YouTube and the Grafana dashboard. A recorded backup of the demo is prepared in case a free LLM API is unavailable.

06 Build plan

The total effort is about 15 working days: roughly 2 days (12-15 hours) per week next to the course.

W1 Repository, secrets cleanup, bundle schema, prompt files, metrics	W2 Docker images, docker compose, FastAPI endpoints	W3 Demand data, DVC, baseline and first model, MLflow registry	W4 MLflow for generation runs, LangChain chains, NGINX	W5 Airflow DAGs, MinIO, review page	W6 YouTube upload, LLM metrics for Prometheus	W7 Grafana, Evidently report, prompt comparison	W8 Minikube, Localstack, GitHub Actions, docs, demo rehearsal
--	---	--	--	---	---	---	---

Colour shows the phase: lightest = Phase 1, darkest = Phase 4.

Risks and fallbacks

Risk	Fallback
Google Trends blocks requests	Collect slowly, cache every answer, keep a versioned snapshot. The demo uses the snapshot.
Free LLM APIs are slow or unavailable	Fallback between providers, pre-generated bundles for the demo, recorded demo video.
Docker images for PDF and video are heavy	One base image with the system packages, built once and reused.
Kubernetes takes too long	Only the API goes to minikube, everything else stays in docker compose.
YouTube API audit not finished in time	Uploads stay private, which is enough for the demo.

07 Directions to explore, by difficulty

Level 1 - this project

One subject, complete chain: weekly run, baseline and one model, quality gates, human approval, YouTube, monitoring.

Level 2 - Scaling up

All grades of Geschichte, Deutsch and Englisch. YouTube search suggestions as an extra signal.

Level 3 - Feedback loop

YouTube Analytics as labels, automated retraining with a promotion rule, monitoring by subject and grade.

Level 4 - Enrichment

Grade-level readability model (from my NLP work) as an extra gate. Local LLM as a free fallback.

08 Metrics

Type	Metric
Business	Time from topic to ready-to-approve material (goal: under 30 minutes without human work) Cost per topic Human rejection rate
Demand model	Spearman rank correlation between predicted and real interest over 4 weeks, compared with the seasonal baseline
LLM quality	Faithfulness score of video texts First-try pass rate of the quality gate Share of valid JSON output
System	Pipeline success rate Run duration API latency (p95)

09 GDPR, security and ethics

Personal data

No student data is collected or sent to LLM providers. Only public curriculum content is used in prompts.

Secrets

API keys in environment variables and Kubernetes secrets, never in git. A secret scan (gitleaks) runs in CI.

AI transparency

Every PDF and video is labelled as AI-assisted and human-reviewed, in line with the EU AI Act (Article 50) and YouTube disclosure settings.

Young audience

Factual errors can harm learning, so quality gates and human approval cannot be skipped. The YouTube "made for kids" setting is chosen honestly.

Copyright

Only openly licensed images with attribution, only fonts with open licences.

Platform rules

YouTube API quota and upload rules are respected.

10 Data and bibliography

Data

- **Saxony curriculum (Lehrplan):**
schulportal.sachsen.de/lplandb
- **Google Trends:** trends.google.com
- **YouTube Data API v3:**
developers.google.com/youtube/v3
- **School holidays:** ferien-api.de
- **Existing AiDemy data:** generated bundles, PDFs, audit reports, videos and SEO metadata

Bibliography

- Sculley et al., Hidden Technical Debt in Machine Learning Systems (NeurIPS 2015)
- Breck et al., The ML Test Score: A Rubric for ML Production Readiness
- Hyndman & Athanasopoulos, Forecasting: Principles and Practice, 3rd ed. - otexts.com/fpp3
- Documentation: [MLflow](#), [LangChain](#), [Evidently](#)
- EU AI Act, Article 50